

Deep Learning

MEEC

2023/2024 – 1^o SEMESTER, P2

Project 2023/24

Teaching:

André Martins
Francisco Melo
João Santinha
Duarte Alves
Margarida Campos

Co-Authors:

Grupo 2
Grupo 14
Grupo 60

Authors:

N.º 100052 – Pedro Apolónia,
N.º 100104 – Tomás Marques

Group 72

Contribution

In this collaborative project, both members of the group actively and equally contributed to every aspect of the work. The collaboration was facilitated through the use of Visual Studio Code's Live Share feature for coding, allowing real-time collaboration and shared development. Additionally, the group utilized Overleaf for collaborative writing, ensuring a seamless integration of code and documentation.

Each member was involved in the project's conception, development, coding, and documentation stages, with an emphasis on joint decision-making and mutual support throughout the entire process. As a result, both members played integral roles, and the distribution of tasks was equitable, reflecting a true team effort in which responsibilities were shared and executed collaboratively.

1 Question 1

1.1 Perceptron

Metric	Value
Train Accuracy	0.4654
Validation Accuracy	0.4610
Final Test Accuracy	0.3422

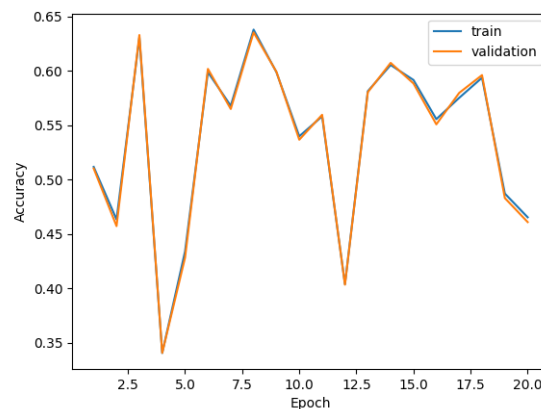
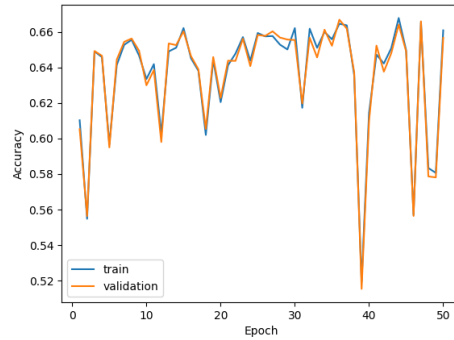


Figure 1: Performance of the Perceptron after training for 20 epochs

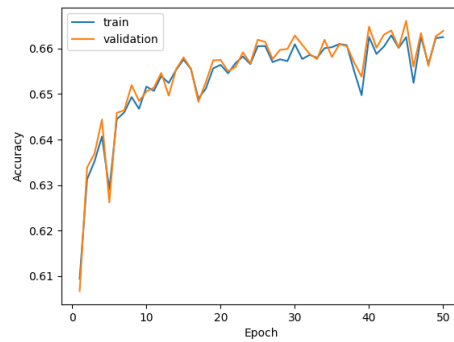
1.2 Logistic Regression

Train Accuracy	0.6609
Validation Accuracy	0.6568
Final Test Accuracy	0.5784

Train Accuracy	0.6625
Validation Accuracy	0.6639
Final Test Accuracy	0.5936



(a) Logistic Regression with $\mu = 0.01$



(b) Logistic Regression with $\mu = 0.001$

Figure 2: Logistic Regression for 50 epoch

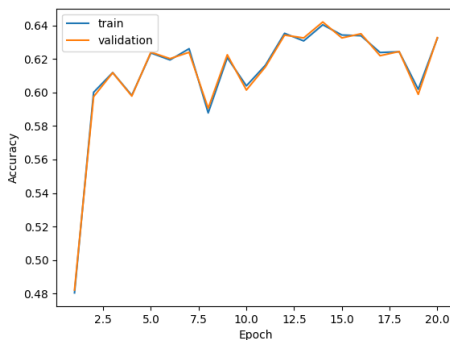
1.3 “A logistic regression model using pixel values as features is not as expressive as a multi-layer perceptron using ReLU activations. However, training a logistic regression model is easier because it is a convex optimization problem.” Is this true or false? Justify.

True. When employing a Multilayer Perceptron (MLP) with Rectified Linear Unit (ReLU) activation for image processing, the model can effectively capture intrinsic relationships between pixels, including various patterns. However, the increased expressiveness of MLPs comes with a trade-off as their optimization is non-convex, presenting challenges in the training process.

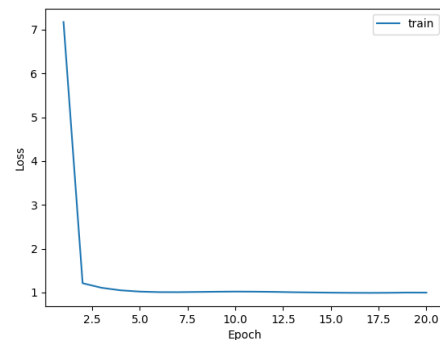
In contrast, Logistic Regression, relying solely on linear pixel values, offers a less expressive yet more straightforward solution. The convex nature of Logistic Regression simplifies the training process compared to the more complex optimization landscape of MLPs with ReLU activation.

1.4 MLP

Loss	0.9969
Train Accuracy	0.6324
Validation Accuracy	0.6328
Final Test Accuracy	0.5917



(a) Train and validation accuracy



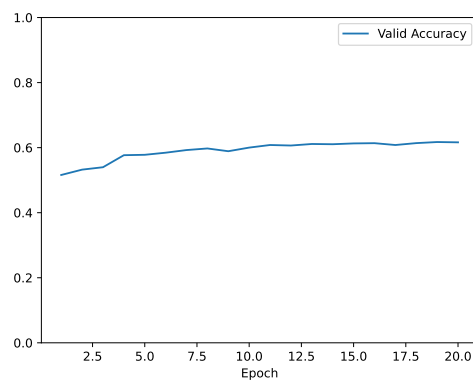
(b) Loss

Figure 3: Performance for 20 epochs with the MLP

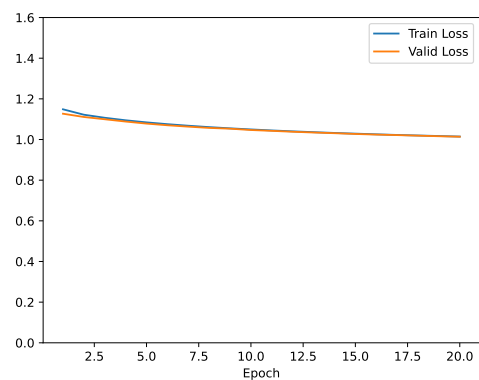
2 Question 2

2.1 Logistic Regression

Training Loss	0.9370
Validation Accuracy	0.6535
Final Test Accuracy	0.6200



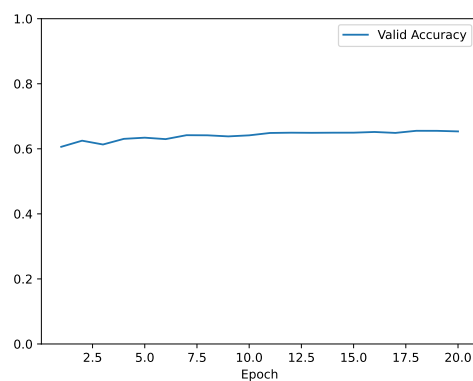
(a) Validation Accuracy



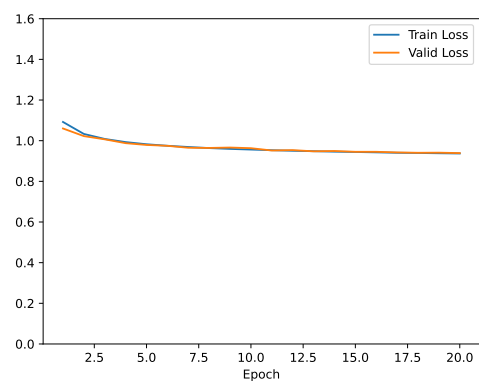
(b) Train and validation Loss

Figure 4: Performance for 20 epochs with logistic regression and Learning rate equal to 0.001

Training Loss	0.9370
Validation Accuracy	0.6535
Final Test Accuracy	0.6200



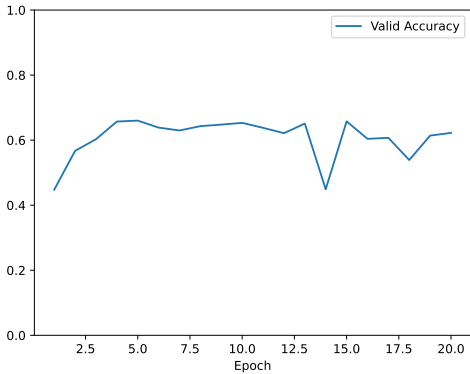
(a) Validation Accuracy



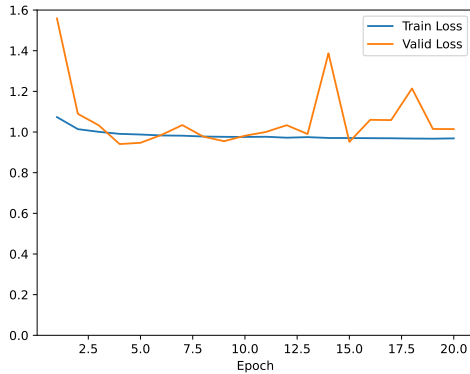
(b) Train and validation Loss

Figure 5: Performance for 20 epochs with logistic regression and Learning rate equal to 0.01

Training Loss	0.9687
Validation Accuracy	0.6224
Final Test Accuracy	0.5577



(a) Validation Accuracy



(b) Train and validation Loss

Figure 6: Performance for 20 epochs with logistic regression and Learning rate equal to 0.1

2.2 MLP a)

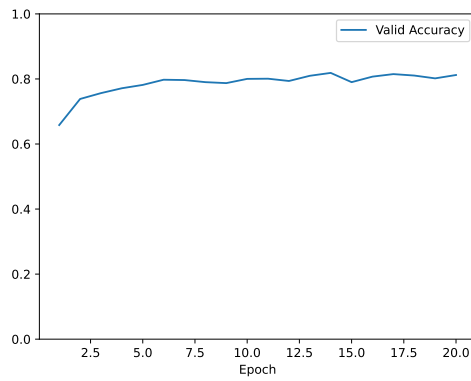
2.2.1 Results Obtained(Batch Size equal to 16)

Training Loss	0.4379
Validation Accuracy	0.8121
Final Test Accuracy	0.7675

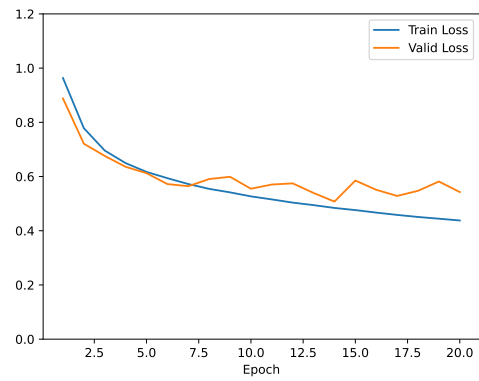
Table 1: Training Results (Epoch 20)

Minutes	2
Seconds	49
Milliseconds	393

Table 2: Time Taken



(a) Validation Accuracy



(b) Train and validation Loss

Figure 7: Performance for 20 epochs with the MLP and batch size 16

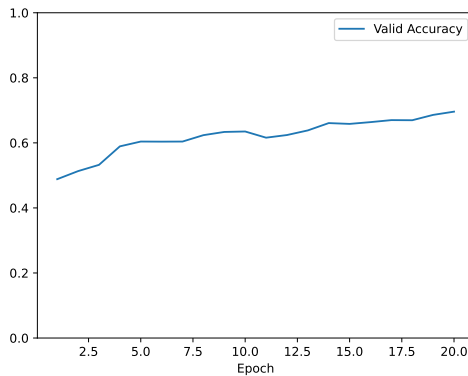
2.2.2 Results Obtained(Batch Size equal to 1024)

Training Loss	0.8713
Validation Accuracy	0.6960
Final Test Accuracy	0.7353

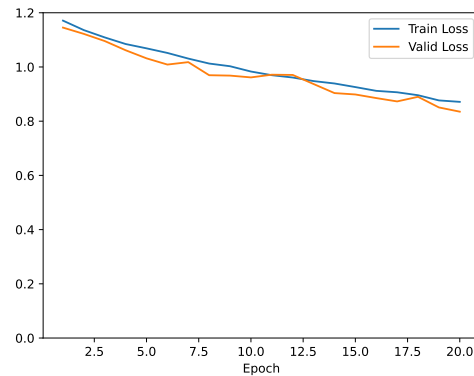
Table 3: Training Results (Epoch 20)

Seconds	41
Milliseconds	385

Table 4: Time Taken



(a) Validation Accuracy



(b) Train and validation Loss

Figure 8: Performance for 20 epochs with the MLP and batch size 1024

2.2.3 Batch size discussion

Comparing model performances with batch sizes of 16 and 1024, the model with a batch size of 16 attained higher accuracy after 20 epochs. However, it's noteworthy that while the 16 batch model plateaued around epoch 8, the 1024 batch model exhibited continued improvement at epoch 20, suggesting the potential for even higher accuracy with additional training epochs.

The observed difference can be attributed to the dynamics of batch size selection. Smaller batch sizes, like 16, allow for more iterations per epoch with fewer samples per iteration, leading to higher accuracy within less epochs. However, results from smaller batch sizes are more susceptible to noise. On the other hand, a larger batch size, such as 1024, results in slower improvement but is less prone to noise, and there is a risk of potential overfitting before reaching a high accuracy.

In conclusion, determining the optimal batch size is nuanced, since the choice between a smaller or larger batch size entails trade-offs. Each has its advantages and disadvantages, and batch size is just one of several factors influencing results. Consideration must be given to the balance between training efficiency and susceptibility to noise, with other factors, such as regularization techniques and model complexity, also playing crucial roles in achieving optimal model performance.

2.2.4 Time of execution discussion

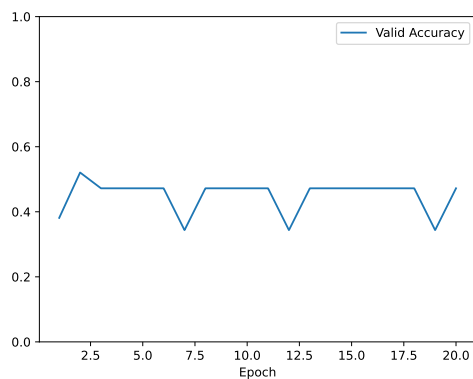
Additionally, it's essential to consider the impact of batch size on the time of execution for the training process. While experimenting with different learning rates, it became evident that the model trained with a larger batch size of 1024 exhibited faster training times compared to the model with a smaller batch size of 16. The larger batch size allows for more efficient parallelization, making better use of available hardware resources.

2.3 MLP b)

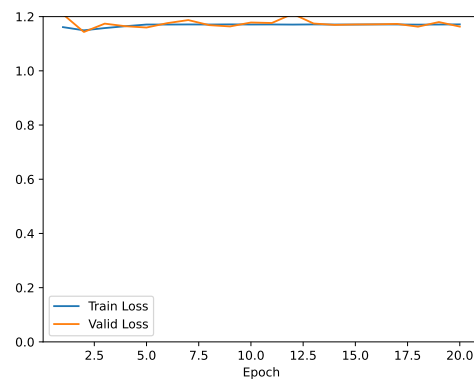
Learning Rate	Best Validation Accuracy
1.0	0.3812
0.1	0.8186
0.01	0.8166
0.001	0.6916

Table 5: Best Validation Accuracies for Each Learning Rate

Training Loss	1.1714
Validation Accuracy	0.4721
Final Test Accuracy	0.4726



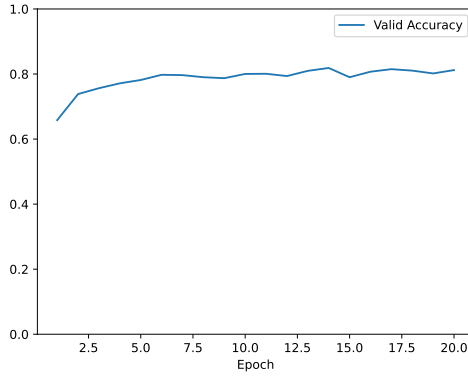
(a) Validation Accuracy



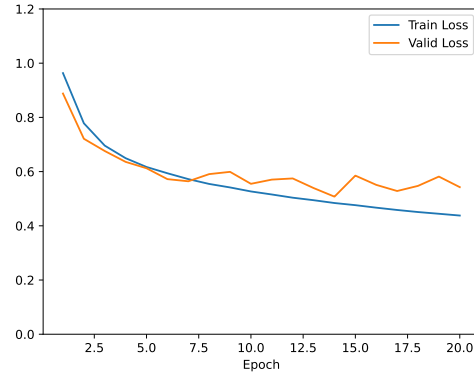
(b) Train and validation Loss

Figure 9: Performance for 20 epochs with the MLP and Learning Rate equal to 1

Training Loss	0.4379
Validation Accuracy	0.8121
Final Test Accuracy	0.7675



(a) Validation Accuracy



(b) Train and validation Loss

Figure 10: Performance for 20 epochs with the MLP and Learning Rate equal to 0.1

2.3.1 Learning rate discussion

After training the model with learning rates of 1, 0.1, 0.01, and 0.001, with the remaining hyperparameters at their default values, distinct patterns in performance emerged. The model trained with a learning rate of 0.1 achieved the highest validation accuracy, while the model with a learning rate of 1 achieved the worst results.

The poor performance of the model with a learning rate of 1 can be attributed to the high likelihood of overshooting the optimal parameter values during training. A learning rate of 1 often causes the model to take excessively large steps in the optimization space, leading to instability, oscillations, and difficulty in converging to an optimal solution.

Conversely, the model with a learning rate of 0.1 struck a balance between convergence speed and avoiding overshooting, resulting in the highest validation accuracy. A moderate learning rate allowed the model to efficiently navigate the optimization landscape, facilitating effective convergence.

It's worth noting that if more epochs were allowed, the models with learning rates of 0.01 or 0.001 might have surpassed the model with a learning rate of 0.1. Smaller learning rates, such as 0.01 and 0.001, often lead to slower but more stable convergence, and given additional training time, these models could potentially reach higher accuracy levels. The choice of the optimal learning rate is, therefore, a delicate balance between achieving satisfactory performance within a limited timeframe and allowing for the model to benefit from more extensive training.

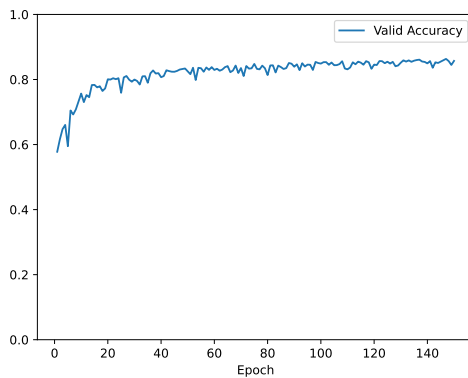
2.4 MLP c)

Technique	Validation Accuracy
Default	0.85
L2 Regularization	0.88
Dropout	0.92

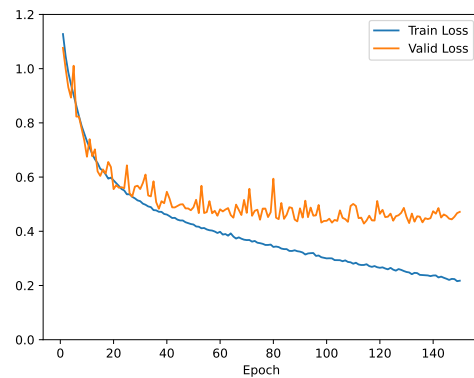
Table 6: Best Validation Accuracy for the 3 models

2.4.1 Default model

Training Loss	0.2177
Validation Accuracy	0.8572
Final Test Accuracy	0.7561



(a) Validation Accuracy



(b) Train and validation Loss

Figure 11: Performance for 150 epochs with the MLP in default mode

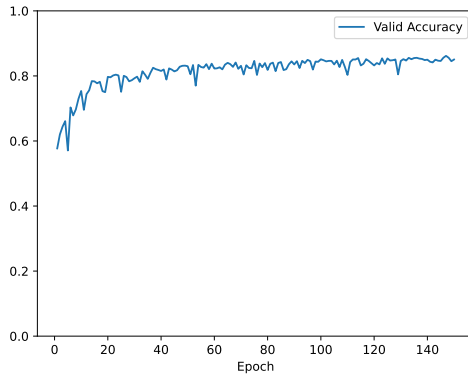
“Using a batch size of 256 run the default model for 150 epochs. Is there overfitting?”

Given the observed behavior where the test loss is decreasing while the validation loss remains constant after a certain epoch, there are indications that the model might be encountering challenges in generalizing to unseen data represented by the validation set. However, it’s important to note that the absence of a decrease in validation loss or improvements in accuracy complicates a definitive characterization of the situation as overfitting.

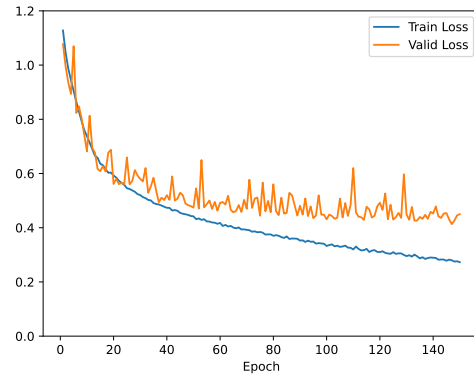
To gain a more comprehensive understanding, further investigation into factors like model complexity, hyperparameter tuning, and potential differences in data distributions between training and validation sets may be warranted. Additionally, monitoring the validation metrics over an extended period or through additional epochs could provide insights into whether the model continues to struggle with generalization.

2.4.2 Default model with l2 paramater regularization

Training Loss	0.2730
Validation Accuracy	0.8504
Final Test Accuracy	0.7486



(a) Validation Accuracy

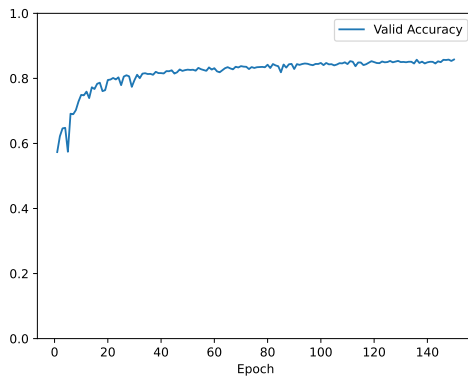


(b) Train and validation Loss

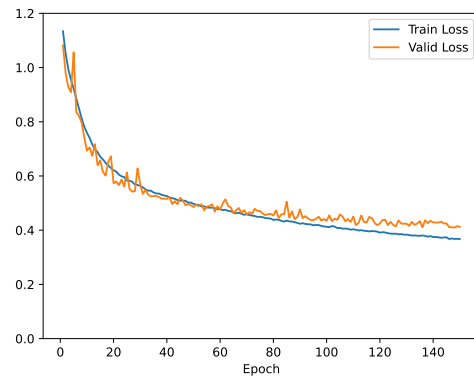
Figure 12: Performance for 150 epochs with the MLP with L2 regularization parameter set to 0.0001

2.4.3 Default model with dropout paramater regularization

Training Loss	0.3671
Validation Accuracy	0.8581
Final Test Accuracy	0.7921



(a) Validation Accuracy



(b) Train and validation Loss

Figure 13: Performance for 150 epochs with the MLP with dropout probability of 0.2

2.4.4 Differences between the two techniques

Dropout introduces a layer of randomness by randomly deactivating a fraction of neurons during each training iteration. This prevents the model from becoming overly reliant on specific neurons, promoting a more robust architecture capable of effective generalization to new data.

In contrast, L2 regularization (or weight decay) imposes a penalty term based on the square of weights in the loss function. This penalty discourages the assignment of excessively large values to weights, promoting smoother optimization and preventing overfitting by penalizing overly complex models.

2.4.5 Discussion of the results

In evaluating their impact on validation accuracy, Dropout demonstrated superior performance compared to L2 regularization, achieving higher accuracy and showcasing a smaller discrepancy between validation and training losses. The analysis emphasized Dropout's significant advantage in maintaining a more consistent and balanced reduction in both validation and training losses. This smaller difference between the loss curves indicates that Dropout not only excelled in preventing overfitting but also facilitated a more harmonious learning process, contributing to enhanced model generalization.

However, it is essential to acknowledge the complementary roles of both techniques. Dropout and L2 regularization play pivotal roles in addressing overfitting, and their effectiveness is contingent on the unique characteristics of the dataset and the specific task at hand. Embracing both regularization methods allows for a more comprehensive approach to model robustness and generalization.

3 Question 3

3.1 a

Consider $D = 2$, and let $A = -1$, $B = 1$. The boolean function f is defined as follows:

$$f(x) = \begin{cases} 1 & \text{if } \sum_{i=1}^D x_i \in [A, B] \\ -1 & \text{otherwise} \end{cases}$$

If $D = 2$, you indeed have two variables, let's say x_1 and x_2 , and each input vector should consist of 2 elements. Therefore, you have four possible combinations for inputs. Let's consider all cases:

1. $x_1 = -1, x_2 = -1 : f(x) = -2$
2. $x_1 = -1, x_2 = +1 : f(x) = 0$
3. $x_1 = +1, x_2 = -1 : f(x) = 0$
4. $x_1 = +1, x_2 = +1 : f(x) = +2$

This will result in the following image:

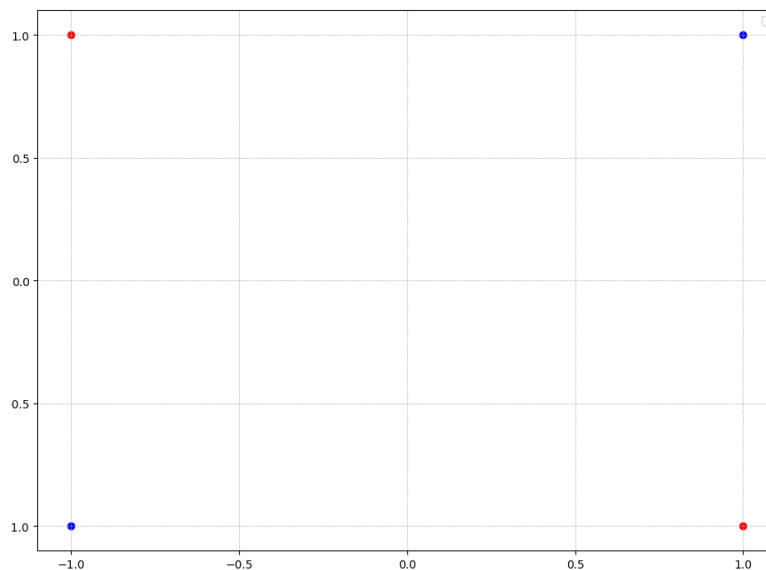


Figure 14: Visualization of $f(x)$ with the red dot at -1 and the blue dot at 1

As evident from the image, the non-linear separability of the function is apparent. Since a single perceptron with only one layer can exclusively create linear decision boundaries, it proves inadequate for addressing this particular complexity.

3.2 b

In order to implement the function $f(x)$, it's possible to achieve this using a single hidden layer with two hidden units. This is accomplished by simultaneously checking if the sum of x is greater than A and smaller than B . By implementing two hidden units, each performing this type of verification, we can effectively implement $f(x)$.

The inequalities to verify in each node of the hidden layer are given by:

$$\begin{cases} \sum_{i=1}^D x_i \geq A \\ \sum_{i=1}^D x_i \leq B \end{cases} \Leftrightarrow \begin{cases} 0 \leq \sum_{i=1}^D x_i - A \\ 0 \leq -\sum_{i=1}^D x_i + B \end{cases}$$

In order to apply this to the hidden layer, it is needed to set weight values and bias values for both nodes. Given that the sum of the inputs is represented by the sum of x , the weights are assigned as follows: 1 for the first node and -1 for the second. For the biases, a bias equal to A is assigned for the first node and B for the second.

This way it is obtained 2 inequalities like before, given by:

$$\begin{cases} 0 \leq w_1 * \sum_{i=1}^D x_i - A \\ 0 \leq -w_2 * \sum_{i=1}^D x_i + B \end{cases}$$

Even though this is true, it is not robust to infinitesimal perturbation of the inputs. Therefore, it is necessary to add a bias of 0.5 to correct these perturbations. However, since 0.5 can't be added to the bias directly (as all weights and biases need to be integers), all the weights and biases are multiplied by 2, resulting in the following inequalities:

$$\begin{cases} 0 \leq 2 * (w_1 * \sum_{i=1}^D x_i - A + 0.5) \\ 0 \leq 2 * (-w_2 * \sum_{i=1}^D x_i + B + 0.5) \end{cases} \Leftrightarrow \begin{cases} 0 \leq 2 * w_1 * \sum_{i=1}^D x_i - 2 * A + 1 \\ 0 \leq -2 * w_2 * \sum_{i=1}^D x_i + 2 * B + 1 \end{cases}$$

This way 2 activation functions are obtained for each node given by:

$$\begin{cases} \hat{y}_1 = \text{sign} \left(2 \cdot w_1 \cdot \sum_{i=1}^D x_i - 2 \cdot A + 1 \right) \\ \hat{y}_2 = \text{sign} \left(-2 \cdot w_2 \cdot \sum_{i=1}^D x_i + 2 \cdot B + 1 \right) \end{cases}$$

For the output layer, it is only needed to check if both nodes return +1, otherwise, the output layer returns -1. To achieve this, all the weights are set to 1 and subtracted by a bias equal to -1. This configuration ensures that the output layer will only return +1 if both hidden nodes return +1.

3.3 c

In the case that the hidden units use ReLU activations instead of the sign, like used before the returned value by the hidden nodes is equal to the max between 0 and $2 * w_1 * \sum_{i=1}^D x_i - 2 * A + 1$ for the 1st node and $2 * w_1 * \sum_{i=1}^D x_i - 2 * A + 1$ for the 2nd node. This can be visualized in Fig15 for both nodes.

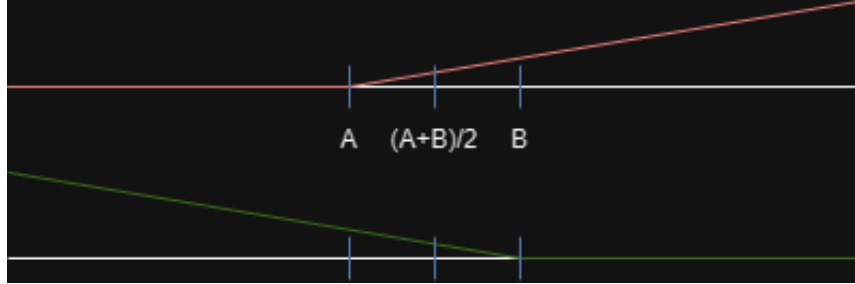


Figure 15: ReLU activation for both hidden nodes

Since we only want our function to activate when both hidden nodes return something positive, we find that by adding a translation of $(A+B)/2$ (Fig16) to both ReLUs and adding them together (Fig17), and then negating what is return by the hidden nodes (Fig18) with a bias of $(B-A)/2$ (Fig19) that we could find a solution that would only activate when both the ReLUs return something positive.

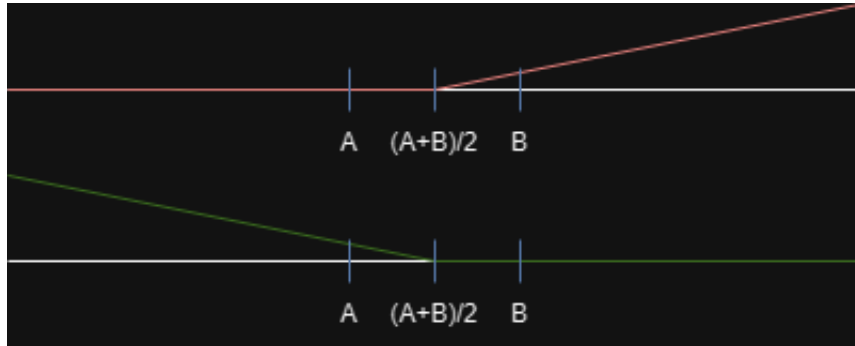


Figure 16: Translation of both functions

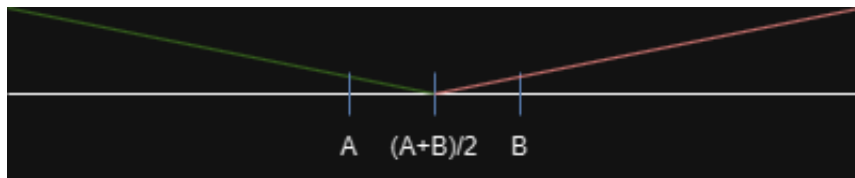


Figure 17: Sum of both functions

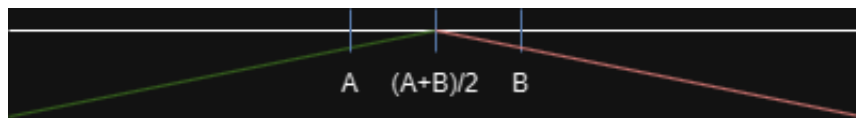


Figure 18: Negation of the functions

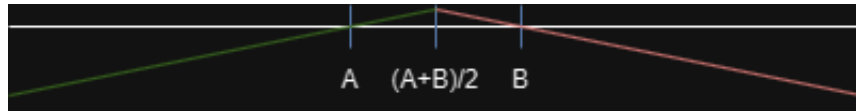


Figure 19: Sum by a constant

This way the hidden nodes would return the functions in Fig17 and with both weights equal to -1 and bias equal to $(B-A)/2$ the output layer would return +1 if the sum of what is returned by the hidden layer is positive.